

集成方法

adaboost算法

算法 8.1 (AdaBoost)

输入: 训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$, 其中 $x_i \in \mathcal{X} \subseteq \mathbf{R}^n$, $y_i \in \mathcal{Y} = \{-1, +1\}$; 弱学习算法;

输出: 最终分类器 $G(x)$ 。

(1) 初始化训练数据的权值分布

$$D_1 = (w_{11}, \dots, w_{1i}, \dots, w_{1N}), \quad w_{1i} = \frac{1}{N}, \quad i = 1, 2, \dots, N$$

(2) 对 $m = 1, 2, \dots, M$

(a) 使用具有权值分布 D_m 的训练数据集学习, 得到基本分类器

$$G_m(x): \mathcal{X} \rightarrow \{-1, +1\}$$

(b) 计算 $G_m(x)$ 在训练数据集上的分类误差率

$$e_m = \sum_{i=1}^N P(G_m(x_i) \neq y_i) = \sum_{i=1}^N w_{mi} I(G_m(x_i) \neq y_i) \quad (8.1)$$

(c) 计算 $G_m(x)$ 的系数

$$\alpha_m = \frac{1}{2} \log \frac{1 - e_m}{e_m} \quad (8.2)$$

这里的对数是自然对数。

(d) 更新训练数据集的权值分布

$$D_{m+1} = (w_{m+1,1}, \dots, w_{m+1,i}, \dots, w_{m+1,N}) \quad (8.3)$$

$$w_{m+1,i} = \frac{w_{mi}}{Z_m} \exp(-\alpha_m y_i G_m(x_i)), \quad i = 1, 2, \dots, N \quad (8.4)$$

这里, Z_m 是规范化因子

$$Z_m = \sum_{i=1}^N w_{mi} \exp(-\alpha_m y_i G_m(x_i)) \quad (8.5)$$

它使 D_{m+1} 成为一个概率分布。

(3) 构建基本分类器的线性组合

$$f(x) = \sum_{m=1}^M \alpha_m G_m(x) \quad (8.6)$$

得到最终分类器

$$\begin{aligned} G(x) &= \text{sign}(f(x)) \\ &= \text{sign}\left(\sum_{m=1}^M \alpha_m G_m(x)\right) \end{aligned} \quad (8.7) \blacksquare$$

逐步回归算法

逐步向前算法

回归: $X^{(1)}, X^{(2)}, \dots, X^{(p)}$ 自变量
 Y 因变量

Stepwise Regression 逐步 { Forward
Backward

$T = \{(x_1, y_1), \dots, (x_n, y_n)\}$

Forward

$Y = \beta_0 + \varepsilon$ 零模型 $f_0(x; \beta)$

$Y = \beta_0 + \beta_1 X^{(1)} + \varepsilon$ $f_1(x; \beta)$

$Y = \beta_0 + \beta_1 X^{(1)} + \beta_2 X^{(2)} + \varepsilon$

AIC, BIC
F检验

逐步向前
前向分步算法



模型从最开始算起，逐步加入因变量 $x^{(j)}$ ，每一次迭代都加入分类效果（判断标准有AIC,BIC）最好的那个剩下的变量（即之前被选择的变量不能再次被选择），直到模型相比于之前没有变得更好，就可以停止了。

算法 8.2 (前向分步算法)

输入: 训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$; 损失函数 $L(y, f(x))$; 基函数集 $\{b(x; \gamma)\}$;

输出: 加法模型 $f(x)$.

(1) 初始化 $f_0(x) = 0$;

(2) 对 $m = 1, 2, \dots, M$

(a) 极小化损失函数

$$(\beta_m, \gamma_m) = \arg \min_{\beta, \gamma} \sum_{i=1}^N L(y_i, f_{m-1}(x_i) + \beta b(x_i; \gamma)) \quad (8.16)$$

得到参数 β_m, γ_m .

(b) 更新

$$f_m(x) = f_{m-1}(x) + \beta_m b(x; \gamma_m) \quad (8.17)$$

(3) 得到加法模型

$$f(x) = f_M(x) = \sum_{m=1}^M \beta_m b(x; \gamma_m) \quad (8.18)$$

逐步后向算法

回归: $X^{(1)}, X^{(2)}, \dots, X^{(p)}$ 自变量
 Y 因变量

Stepwise Regression 逐步 { Forward
 Backward

$T = \{(x_1, y_1), \dots, (x_N, y_N)\}$

Backward

$Y = \beta_0 + \beta_1 X^{(1)} + \dots + \beta_p X^{(p)} + \varepsilon$ 全 $f_p(x; \beta)$

$Y = \beta_0 + \beta_1 X^{(1)} + \dots + \beta_{p-1} X^{(p-1)} + \varepsilon$ $f_{p-1}(x; \beta)$

剔除
 踢 out

每次剔除一个因变量，选择一个剔除后能让模型变好的幅度最大的，直到不可能因为剔除变量而变得更好为止。（类似于剪枝）

提升树

算法 8.2 (前向分步算法)

输入: 训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$; 损失函数 $L(y, f(x))$; 基函数集 $\{b(x; \gamma)\}$

输出: 加法模型 $f(x)$.

(1) 初始化 $f_0(x) = 0$; ✓

(2) 对 $m = 1, 2, \dots, M$

(a) 极小化损失函数

$$\hat{\theta}_m = \underset{\theta_m}{\operatorname{argmin}} \sum_{i=1}^N L(y_i, f_{m-1}(x) + \theta_m b(x; \gamma))$$

$$(\beta_m, \gamma_m) = \underset{\beta, \gamma}{\operatorname{argmin}} \sum_{i=1}^N L(y_i, f_{m-1}(x) + \beta b(x; \gamma)) \quad (8.16)$$

得到参数 β_m, γ_m .

(b) 更新

$$f_m(x) = f_{m-1}(x) + \beta_m b(x; \gamma_m) \quad (8.17)$$

(c) 得到加法模型

$$f_M(x) = \sum_{m=1}^M \beta_m b(x; \gamma_m) \quad (8.18)$$

提升树

- 二叉树 ✓
- 可加模型
- 前向分步算法

输入: $T; L(y, f(x))$ { 分类: 指数; 回归: 平方; }

$T(x; \theta_m)$

输出: $f_m(x)$



基于前向分步算法。得到对应的提升树算法。最终求和如果是分类任务，采用众数；如果是回归任务可

以采用平均数，中位数等。

注意决策树做回归任务时，依旧是按 x 的值进行分类的，只不过每个 x 会算出不同的 $f(x)$ ，所以做平均值的时候，应该是同一分类下的值做平均；算损失函数的时候，应该是 $f(x) - T(x)$ ，其中 $T(x)$ 是决策树按照对应 x 得到的结果（即这一类的均值）。

算法 8.3 (回归问题的提升树算法)

输入：训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$, $x_i \in \mathcal{X} \subseteq \mathbb{R}^n$, $y_i \in \mathcal{Y} \subseteq \mathbb{R}$

输出：提升树 $f_M(x)$ 。

(1) 初始化 $f_0(x) = 0$ 。

(2) 对 $m = 1, 2, \dots, M$ 。

(a) 按式 (8.27) 计算残差：

$$r_{mi} = y_i - f_{m-1}(x_i), \quad i = 1, 2, \dots, N$$

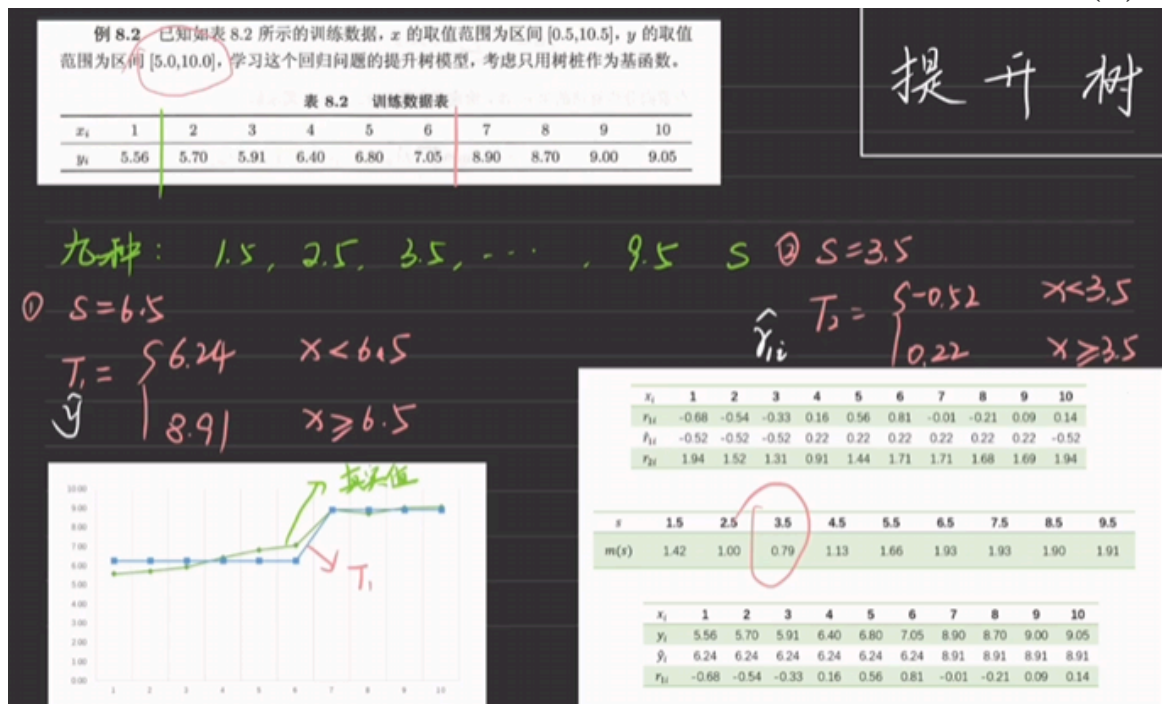
(b) 拟合残差 r_{mi} 学习一个回归树，得到 $T(x; \Theta_m)$ 。

(c) 更新 $f_m(x) = f_{m-1}(x) + T(x; \Theta_m)$ 。

(3) 得到回归问题提升树

$$f_M(x) = \sum_{m=1}^M T(x; \Theta_m)$$

具体而言我们可以使用前一次得到的误差项得到一个回归树，作为新一轮的 $f(x)$ ，例如：



梯度提升算法

算法 8.4 (梯度提升算法)

输入: 训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$, $x_i \in \mathcal{X} \subseteq \mathbb{R}^n$, $y_i \in \mathcal{Y} \subseteq \mathbb{R}$
损失函数 $L(y, f(x))$;
输出: 回归树 $\hat{f}(x)$.

(1) 初始化

$$f_0(x) = \arg \min_c \sum_{i=1}^N L(y_i, c)$$

(2) 对 $m = 1, 2, \dots, M$

(a) 对 $i = 1, 2, \dots, N$, 计算

$$r_{mi} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f(x)=f_{m-1}(x)}$$

(b) 对 r_{mi} 拟合一个回归树, 得到第 m 棵树的叶结点区域 R_{mj} , $j = 1, 2, \dots, J$.

(c) 对 $j = 1, 2, \dots, J$, 计算

$$c_{mj} = \arg \min_c \sum_{x_i \in R_{mj}} L(y_i, f_{m-1}(x_i) + c)$$

(d) 更新 $f_m(x) = f_{m-1}(x) + \sum_{j=1}^J c_{mj} I(x \in R_{mj})$

(3) 得到回归树

$$\hat{f}(x) = f_M(x) = \sum_{m=1}^M \sum_{j=1}^J c_{mj} I(x \in R_{mj})$$